NMAM INSTITUTE OF TECHNOLOGY, NITTE

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

Concurrency control using semaphores, monitors, and live Java–browser execution

Student name: _____

USN: _____

Course: Operating Systems

Submitted to: _____

Date: _____

Submission note: Complete the student information after reviewing the implementation and rerunning the programs locally.

1. Problem Statement and Objectives

1.1 Problem context

Several threads may contend for the same resource. Without a coordination rule, one thread can change shared data while another reads it, or multiple threads can become stuck waiting for one another. This assignment demonstrates both problems using four independently selectable Java implementations. [1,2]

1.2 Objectives

- Represent four philosophers and four chopsticks as threads and shared synchronization resources.
- Prevent deadlock during philosopher resource acquisition, using two different strategies.
- Permit simultaneous readers but enforce exclusive writer access.
- Use array-based resource mapping and an explicit FIFO request queue where appropriate.
- Connect real Java execution to a browser visualization and preserve timestamped event logs.
- Check safety invariants against actual recorded executions.

2. Readers–Writers: Semaphore-Based Solution

2.1 Design decision

Five reader threads and three writer threads access one integer initialized to 10. Three fair Java semaphores serve separate roles: turnstile (entry fairness), roomEmpty (exclusive resource), and countLock (reader-count protection). The first active reader takes roomEmpty, and the last one releases it. A writer takes turnstile before roomEmpty, preventing newly arriving readers from continually overtaking it. [3]

2.2 Algorithm and data structure

Reader: pass the turnstile; update reader count under countLock; if first, acquire roomEmpty; read; decrement under countLock; if last, release roomEmpty. Writer: acquire turnstile, acquire roomEmpty, write, release roomEmpty, release turnstile. Storage includes Semaphore objects, an AtomicInteger for reader count, and an AtomicInteger for shared value.

2.3 Expected properties

Multiple readers can overlap. A writer cannot overlap any reader or another writer. The fair turnstile reduces writer starvation for this finite workload, although thread scheduling does not give hard execution-time guarantees.

3. Readers–Writers: Monitor-Based Solution

3.1 Design decision

This variant does not reuse the semaphore routine. It uses synchronized methods, wait(), and notifyAll() around an ArrayDeque request queue. Waiting threads register identifiers such as R0

and W0. A reader can enter if no writer is active and there is no writer ahead of it in the queue. A writer can enter only at the head of the queue when the resource is completely free. [2,4]

3.2 Queue-based admission

The request queue adds a meaningful data-structure component. Reader requests before a writer may be admitted together; a later reader cannot bypass an already queued writer. Operations use Java intrinsic monitor locking to protect both the queue and active counts. These choices make ordering easy to explain during a viva.

4. Dining Philosophers: Semaphore-Based Solution

4.1 Mapping and deadlock avoidance

Four philosopher threads share four fair binary semaphores stored in an array. For philosopher p, left = p and right = (p + 1) % 4. Even IDs acquire left then right. Odd IDs acquire right then left. This asymmetric ordering breaks a circular chain of waiting, so the classic all-left-hand deadlock is avoided. [1,3]

4.2 Execution flow

Think → become hungry → acquire first fork → acquire second fork → eat → release both forks → finish the round. Each philosopher performs three rounds. Non-neighboring philosophers may eat at the same time. Semaphore fairness on individual forks does not establish a formal starvation guarantee for the whole algorithm.

5. Dining Philosophers: Monitor-Based Solution

5.1 Atomic pair allocation

A ForkMonitor keeps a boolean array inUse[4]. Its synchronized acquire method waits while either required fork is occupied. When both are free, the monitor marks them occupied in the same critical section. After eating, the thread marks both free and calls notifyAll(). The philosopher never holds one fork while waiting for the second. [2,4]

5.2 Benefit and limitation

This design demonstrates a contrasting policy: the semaphore version breaks circular wait by acquisition order, while the monitor version avoids partial allocations altogether. Both preserve safe shared-fork ownership. Neither variant claims strict fairness under unbounded workloads.

6. Browser-Connected Execution and Visualization

The application is a Java HTTP server using the JDK built-in HttpServer API. The browser serves HTML, CSS, and JavaScript from the same Java process. Clicking Start sends POST /api/run?mode=<selected-mode>. The Java backend starts actual threads and writes serialized, timestamped newline-delimited JSON events to the HTTP response. JavaScript reads the response

stream with fetch(), updates thread/fork states, animates blue active resources, and appends the event timeline. An Export Log button produces CSV. No frontend animation engine pretends to run the algorithms independently.

The interface uses black/navy backgrounds, electric-blue highlights, state cards, a circular philosopher table, and a shared-data display. Only one simulation executes at once to keep the log understandable.

7. Actual Runs, Observations, and Evidence

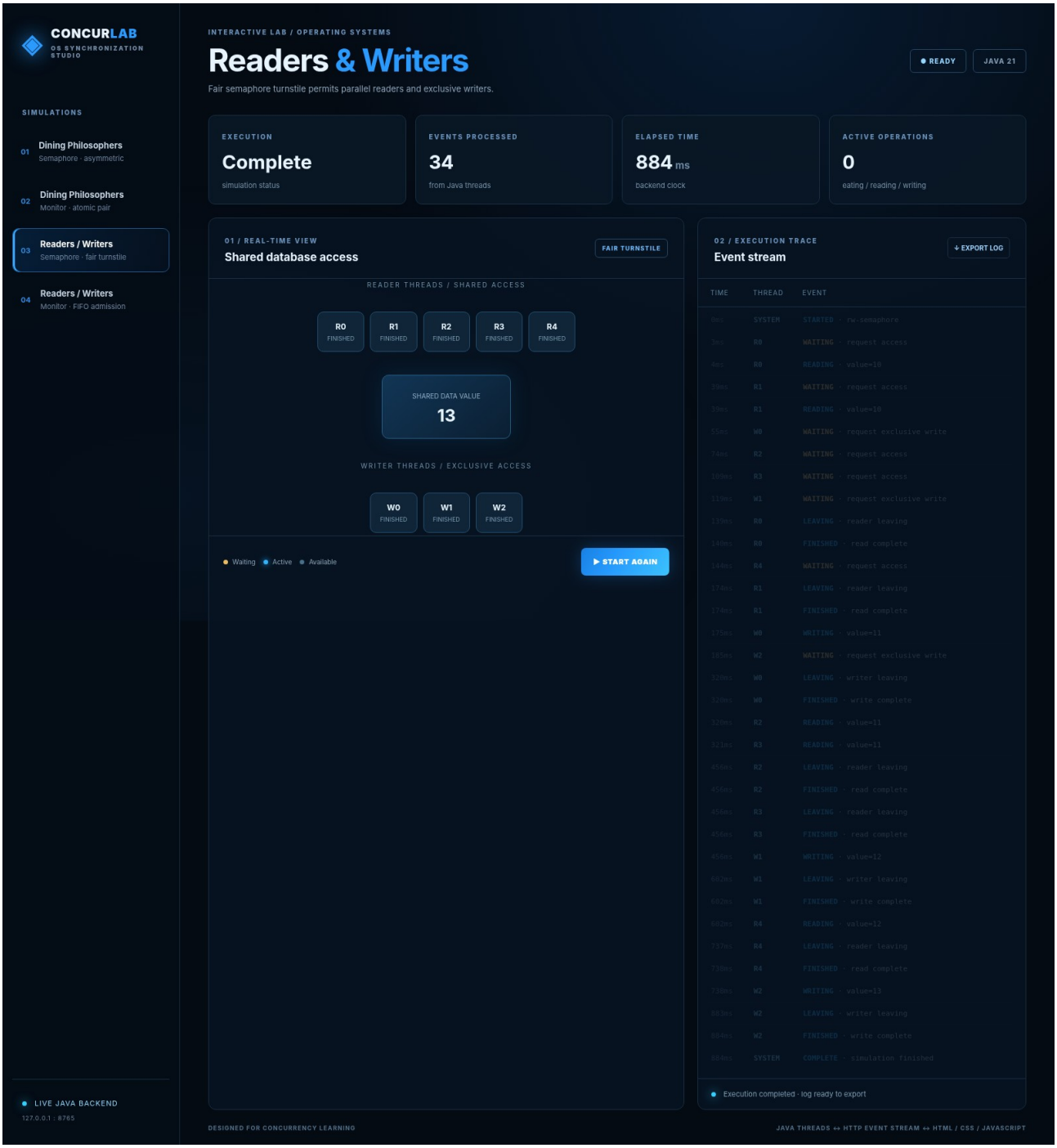
The Java 21 source compiled successfully using javac. Four POST requests to the local Java backend produced the recorded .jsonl files listed below. Measured times include intentional sleep delays used to make thread interaction observable; they do not compare raw algorithm speed.

Implementation	Events	Elapsed (ms)	Completed operations
philosophers-semaphore	110	845	12 eating rounds
philosophers-monitor	110	757	12 eating rounds
rw-semaphore	34	884	5 reads, 3 writes
rw-monitor	26	882	5 reads, 3 writes

Automated log verification passed for all four recorded runs: sequential event numbers, start/completion markers, non-overlapping neighboring eaters, and no simultaneous writer/reader or writer/writer activity.

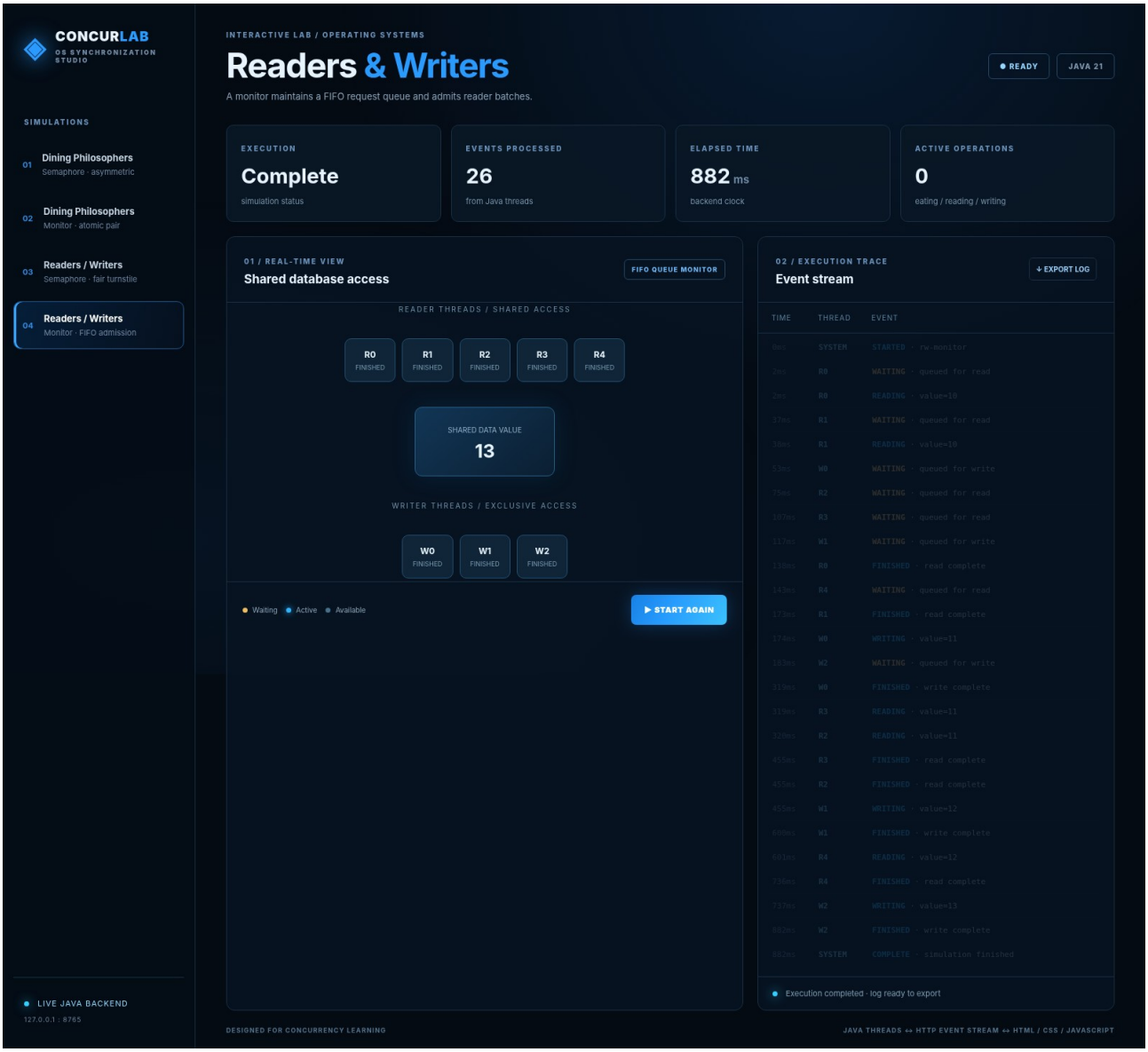
Image evidence note: Because the browser test environment restricted direct localhost navigation, the screenshots below were rendered from the original interface using replay of captured Java event traces. The trace files themselves are genuine Java-backend execution outputs. Rerun the app on your laptop and take live screenshots for final submission.

7. Readers–Writers with semaphore



Screenshot: Readers–Writers with semaphore, event-log replay.

7. Readers–Writers with monitor



Screenshot: Readers–Writers with monitor, event-log replay.

7. Dining Philosophers with semaphore

CONCURLAB
OS SYNCHRONIZATION STUDIO

SIMULATIONS

01 **Dining Philosophers**
Semaphore · asymmetric

02 **Dining Philosophers**
Monitor · atomic pair

03 **Readers / Writers**
Semaphore · fair turnstile

04 **Readers / Writers**
Monitor · FIFO admission

INTERACTIVE LAB / OPERATING SYSTEMS

Dining Philosophers

Deadlock-free resource coordination through asymmetric semaphore acquisition.

● READY | JAVA 21

EXECUTION	EVENTS PROCESSED	ELAPSED TIME	ACTIVE OPERATIONS
Complete	110	845 ms	0
simulation status	from Java threads	backend clock	eating / reading / writing

01 / REAL-TIME VIEW
Resource allocation

ASYMMETRIC SEMAPHORE



Waiting · Active · Available

▶ START AGAIN

02 / EXECUTION TRACE
Event stream

↓ EXPORT LOG

TIME	THREAD	EVENT
2ms	SYSTEM	STARTED · philosophers-semaphore
15ms	P0	THINKING · round=1
15ms	P2	THINKING · round=1
15ms	P1	THINKING · round=1
16ms	P3	THINKING · round=1
81ms	P0	HUNGRY · waiting for chopsticks
82ms	F0	HELD · P0
82ms	F1	HELD · P0
82ms	P0	EATING · round=1
98ms	P1	HUNGRY · waiting for chopsticks
98ms	F2	HELD · P1
116ms	P2	HUNGRY · waiting for chopsticks
132ms	P3	HUNGRY · waiting for chopsticks
187ms	P0	RELEASING · returning chopsticks
188ms	F1	HELD · P1
188ms	P1	EATING · round=1
188ms	F1	FREE · P0
188ms	F0	FREE · P0
189ms	P0	FINISHED · round=1
189ms	P0	THINKING · round=2
189ms	F0	HELD · P3
189ms	F3	HELD · P3
189ms	P3	EATING · round=1
254ms	P0	HUNGRY · waiting for chopsticks
293ms	P1	RELEASING · returning chopsticks
293ms	F1	FREE · P1
293ms	F2	FREE · P1
293ms	P1	FINISHED · round=1
293ms	F2	HELD · P2
294ms	P1	THINKING · round=2
294ms	P3	RELEASING · returning chopsticks
294ms	F3	FREE · P3
295ms	F3	HELD · P2
295ms	P2	EATING · round=1
295ms	F0	HELD · P0
295ms	F1	HELD · P0
295ms	P0	EATING · round=2
295ms	F0	FREE · P3
295ms	P3	FINISHED · round=1
295ms	P3	THINKING · round=2
376ms	P1	HUNGRY · waiting for chopsticks
400ms	P2	RELEASING · returning chopsticks
400ms	F3	FREE · P2
400ms	F2	FREE · P2
400ms	F2	HELD · P1
400ms	P0	RELEASING · returning chopsticks
401ms	P2	FINISHED · round=1
401ms	P2	THINKING · round=2
401ms	F1	HELD · P1
401ms	P1	EATING · round=2
401ms	F1	FREE · P0
401ms	F0	FREE · P0
401ms	P0	FINISHED · round=2
401ms	P0	THINKING · round=3

Screenshot: Dining Philosophers with semaphore, event-log replay.

7. Dining Philosophers with monitor

CONCURLAB
OS SYNCHRONIZATION STUDIO

SIMULATIONS

01 Dining Philosophers
Semaphore · asymmetric

02 Dining Philosophers
Monitor · atomic pair

03 Readers / Writers
Semaphore · fair turnstile

04 Readers / Writers
Monitor · FIFO admission

INTERACTIVE LAB / OPERATING SYSTEMS

Dining Philosophers

Atomic chopstick allocation using a synchronized Java monitor.

● READY | JAVA 21

EXECUTION	EVENTS PROCESSED	ELAPSED TIME	ACTIVE OPERATIONS
Complete	110	757 ms	0
simulation status	from Java threads	backend clock	eating / reading / writing

01 / REAL-TIME VIEW
Resource allocation

ATOMIC PAIR MONITOR



Waiting · Active · Available

▶ START AGAIN

02 / EXECUTION TRACE
Event stream

↓ EXPORT LOG

TIME	THREAD	EVENT
0ms	SYSTEM	STARTED · philosophers-monitor
2ms	P0	THINKING · round=1
2ms	P1	THINKING · round=1
2ms	P2	THINKING · round=1
2ms	P3	THINKING · round=1
67ms	P0	HUNGRY · atomic pair requested
68ms	F0	HELD · P0
68ms	F1	HELD · P0
68ms	P0	EATING · round=1
84ms	P1	HUNGRY · atomic pair requested
101ms	P2	HUNGRY · atomic pair requested
101ms	F2	HELD · P2
102ms	F3	HELD · P2
102ms	P2	EATING · round=1
118ms	P3	HUNGRY · atomic pair requested
174ms	P0	RELEASING · returning chopsticks
174ms	F0	FREE · P0
174ms	F1	FREE · P0
175ms	P0	FINISHED · round=1
175ms	P0	THINKING · round=2
207ms	P2	RELEASING · returning chopsticks
207ms	F2	FREE · P2
207ms	F3	FREE · P2
207ms	P2	FINISHED · round=1
207ms	P2	THINKING · round=2
207ms	F1	HELD · P1
207ms	F2	HELD · P1
207ms	P1	EATING · round=1
208ms	F3	HELD · P3
208ms	F0	HELD · P3
208ms	P3	EATING · round=1
240ms	P0	HUNGRY · atomic pair requested
307ms	P2	HUNGRY · atomic pair requested
313ms	P1	RELEASING · returning chopsticks
313ms	F1	FREE · P1
313ms	F2	FREE · P1
313ms	P1	FINISHED · round=1
313ms	P1	THINKING · round=2
313ms	P3	RELEASING · returning chopsticks
313ms	F3	FREE · P3
313ms	F0	FREE · P3
313ms	F0	HELD · P0
313ms	F1	HELD · P0
313ms	P0	EATING · round=2
313ms	F2	HELD · P2
313ms	F3	HELD · P2
313ms	P2	EATING · round=2
313ms	P3	FINISHED · round=1
313ms	P3	THINKING · round=2
395ms	P1	HUNGRY · atomic pair requested
418ms	P0	RELEASING · returning chopsticks
419ms	P2	RELEASING · returning chopsticks
419ms	F0	FREE · P0
419ms	F1	FREE · P0

Screenshot: Dining Philosophers with monitor, event-log replay.

8. Synchronization Concepts Demonstrated

- Critical section: code whose shared state must be protected.
- Deadlock: a cycle of threads unable to proceed because each waits on resources held by another.
- Starvation: a thread waits indefinitely while other requests keep getting served.
- Semaphore: a permit-based object acquired and released explicitly.
- Monitor: a synchronized object whose state and wait conditions are encapsulated together.
- Thread: independently scheduled execution inside one Java process; threads share the arrays, queues, and counters.

9. Semaphore versus Monitor

Criterion	Semaphore variants	Monitor variants
Resource control	Explicit acquire/release	Synchronized methods and condition wait
Dining solution	Odd/even ordered binary semaphores	Atomic allocation of both forks
Readers–Writers solution	Fair turnstile + shared reader count	FIFO queue + admission predicates
Main data structures	Semaphore[4], counters	boolean[4], ArrayDeque requests
Fairness caveat	No absolute scheduling guarantee	Queued admission for RW; DP may starve

10. What Was Learned

The demonstrations show why concurrency correctness depends on resource ownership and admission rules rather than on thread creation alone. A small design change—altering fork-acquisition order or enforcing atomic allocation—can remove a specific deadlock scenario. The reader/writer experiments also show why a counter must be protected, why an exclusive gate is needed for writes, and how a FIFO request queue makes fairness more explicit. The live Java-to-browser event stream makes synchronization behavior easier to inspect than a final console message.

11. Assistance and Integrity Note

This project was developed with AI-assisted drafting and coding support, followed by actual Java compilation, execution, and trace validation. The included code and narrative are custom to this design rather than copied from the example report. Before academic submission, the student

should run the app personally, inspect the source, replace placeholder identity fields, capture local screenshots, and disclose assistance according to the institution's requirements. No claim is made about Turnitin similarity or AI-detection outcomes.

12. References

[1] Silberschatz, Galvin, and Gagne, Operating System Concepts, 10th ed., Wiley, 2018. https://codex.cs.yale.edu/avi/os-book/OS10/

[2] Oracle, Java Tutorials — Synchronization. https://docs.oracle.com/javase/tutorial/essential/concurrency/sync.html

[3] Oracle, Java SE — Semaphore API. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/Semaphore.html

[4] Oracle, Java Language Specification, Chapter 17: Threads and Locks. https://docs.oracle.com/javase/specs/jls/se21/html/jls-17.html

[5] Oracle, Java SE — HttpServer. https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html

13. Student Declaration

I have reviewed the code and verification evidence, executed the work myself, and acknowledge all outside assistance and consulted sources as required by my institution.

Name: _____ Signature: _____ Date: _____